Generative AI Course Project — Ranked Idea Compendium

Semester 7 B.Tech · Prepared 2026-09-14

Constraints locked in for this list

- Medium–hard difficulty, resume-worthy — not a beginner project
- Text / LLM only — no image-based work
- No readymade datasets — the dataset must be self-collected
- Hardware: Colab free / Kaggle tier — T4 GPU, 16GB VRAM, session disconnects
- Graded by a **live demo to a panel of judges**

20 ideas surveyed across four rounds of narrowing. Ideas that violate a constraint are kept for completeness in the Appendix, clearly flagged — not silently dropped.

How to read this

Three independent 1–5 ratings per idea. **Complexity** = engineering + conceptual difficulty at your hardware tier. **Resume value** = how much it stands out in a screening pass and survives a follow-up question in an interview. **Demo value** = how well it lands in a live run in front of judges — panels reward *visible contrast* (baseline fails, yours succeeds, understood in five seconds without reading a chart), not output quality and not implementation sophistication. Demo value is orthogonal to the other two and can invert them: D1 rates highest for resume value and lowest for demo value — its output is visibly weak, and a panel grades what it sees. Three compliance flags per idea: text-only, self-collected data, T4-feasible. All three checked = fully compliant with your current constraints.

Table A — Ranked by Complexity

#	Idea	Complexity	Resume	Demo	Text-only	Self-collected	T4-OK
D1	Discrete text diffusion vs. AR twin	**5**	**5**	**2**	✓	✓	✓*
A7	DPO from scratch	**4.5**	**4**	**2.5**	✓	✗	✓*
A6	Speculative decoding from scratch	**4.5**	**3.5**	**4.5**	✓	✗	✓
C1	Verified synthetic distillation	**4.5**	**4.5**	**4**	✓	✓	✓
C2	Hinglish / code-mixed eval set	**4**	**4.5**	**4**	✓	✓	✓
C3	GitHub code-review mining	**4**	**4**	**3**	✓	✓	✓
C5	Prompt-injection corpus	**4**	**4.5**	**5**	✓	✓	✓
R1	Temporal RAG (Wikipedia revisions)	**4**	**5**	**5**	✓	✓	✓
R2	RAG corpus poisoning attack/defense	**4**	**4.5**	**5**	✓	✓	✓
R3	Multi-hop domain RAG (self-scraped)	**4**	**3.5**	**3.5**	✓	✓	✓

A2	Self-repairing code agent	4	4	3.5	✓	✗	✓
A4	LLM watermarking + attacks	4	5	3.5	✓	✗†	✓
A5	Automated red-teaming harness	4	4.5	4	✓	✗†	✓
A8	GraphRAG on HotpotQA	4	3.5	3	✓	✗†	✓
D2	Tabular diffusion + membership inference	4	4	3	✓	✓	✓
C4	Post-cutoff hallucination benchmark	3.5	4	3.5	✓	✓	✓
C6	Cross-generator AI-text detector	3.5	4	4.5	✓	✓	✓
A9	Self-consistency hallucination detection	3.5	4	3	✓	✗†	✓
B1	Image diffusion from scratch	4	3	3	✗	✓	**tight**
B2	Sketch → working artifact	4	5	4.5	✗	✓	✓

* D1 and A7 are feasible on a free T4 only by staying small (D1: ~30–60M params; A7: QLoRA on a 3B model) — see catalogue entries.
† Marked ✗ only for the readymade dataset originally cited (C4/HotpotQA, hh-rlhf, AdvBench, TruthfulQA). Each can be rescued with a self-collected substitute — noted inline below.

Table B — Ranked by Resume Value

#	Idea	Resume	Complexity	Demo	Text-only	Self-collected	T4-OK
D1	Discrete text diffusion vs. AR twin	5	5	2	✓	✓	✓*
R1	Temporal RAG (Wikipedia revisions)	5	4	5	✓	✓	✓
A4	LLM watermarking + attacks	5	4	3.5	✓	✗†	✓
B2	Sketch → working artifact	5	4	4.5	✗	✓	✓
C1	Verified synthetic distillation	4.5	4.5	4	✓	✓	✓
C2	Hinglish / code-mixed eval set	4.5	4	4	✓	✓	✓
C5	Prompt-injection corpus	4.5	4	5	✓	✓	✓
R2	RAG corpus poisoning attack/defense	4.5	4	5	✓	✓	✓
A5	Automated red-teaming harness	4.5	4	4	✓	✗†	✓
A7	DPO from scratch	4	4.5	2.5	✓	✗	✓*
A2	Self-repairing code agent	4	4	3.5	✓	✗	✓
C3	GitHub code-review mining	4	4	3	✓	✓	✓
C4	Post-cutoff hallucination benchmark	4	3.5	3.5	✓	✓	✓

C6	Cross-generator AI-text detector	**4**	**3.5**	**4.5**	✓	✓	✓
D2	Tabular diffusion + membership inference	**4**	**4**	**3**	✓	✓	✓
A9	Self-consistency hallucination detection	**4**	**3.5**	**3**	✓	✗†	✓
A6	Speculative decoding from scratch	**3.5**	**4.5**	**4.5**	✓	✗	✓
R3	Multi-hop domain RAG (self-scraped)	**3.5**	**4**	**3.5**	✓	✓	✓
A8	GraphRAG on HotpotQA	**3.5**	**4**	**3**	✓	✗†	✓
B1	Image diffusion from scratch	**3**	**4**	**3**	✗	✓	**tight**

Table C — Ranked by Demo Value

This is now the decision-driving axis. A project can score high here purely by staging a moment a panel understands in five seconds — regardless of how hard it was to build.

#	Idea	Demo	Complexity	Resume	Text-only	Self-collected	T4-OK
R1	Temporal RAG (Wikipedia revisions)	**5**	**4**	**5**	✓	✓	✓
R2	RAG corpus poisoning attack/defense	**5**	**4**	**4.5**	✓	✓	✓
C5	Prompt-injection corpus	**5**	**4**	**4.5**	✓	✓	✓
A6	Speculative decoding from scratch	**4.5**	**4.5**	**3.5**	✓	✗	✓
C6	Cross-generator AI-text detector	**4.5**	**3.5**	**4**	✓	✓	✓
B2	Sketch → working artifact	**4.5**	**4**	**5**	✗	✓	✓
C1	Verified synthetic distillation	**4**	**4.5**	**4.5**	✓	✓	✓
C2	Hinglish / code-mixed eval set	**4**	**4**	**4.5**	✓	✓	✓
A5	Automated red-teaming harness	**4**	**4**	**4.5**	✓	✗†	✓
R3	Multi-hop domain RAG (self-scraped)	**3.5**	**4**	**3.5**	✓	✓	✓
C4	Post-cutoff hallucination benchmark	**3.5**	**3.5**	**4**	✓	✓	✓
A4	LLM watermarking + attacks	**3.5**	**4**	**5**	✓	✗†	✓
A2	Self-repairing code agent	**3.5**	**4**	**4**	✓	✗	✓
C3	GitHub code-review mining	**3**	**4**	**4**	✓	✓	✓
D2	Tabular diffusion + membership inference	**3**	**4**	**4**	✓	✓	✓
A8	GraphRAG on HotpotQA	**3**	**4**	**3.5**	✓	✗†	✓

A9	Self-consistency hallucination detection	**3**	**3.5**	**4**	✓	✗†	✓
B1	Image diffusion from scratch	**3**	**4**	**3**	✗	✓	**tight**
A7	DPO from scratch	**2.5**	**4.5**	**4**	✓	✗	✓*
D1	Discrete text diffusion vs. AR twin	**2**	**5**	**5**	✓	✓	✓*

Demo Tiers — Staging Notes for a Live Panel

Tier 1 — the demo sells itself

- **R2 · RAG poisoning.** Ask the question, get the correct answer. Inject three poison documents live. Ask again — the system now confidently states the false answer. Toggle the defense on, correct answer returns. Three visible state changes in ~90 seconds, no interpretation required.
- **R1 · Temporal RAG.** Side-by-side panes, identical time-qualified question. Vanilla RAG confidently returns the superseded clause; yours returns the current one with a revision timeline rendered underneath. The judge sees *why* the baseline is wrong, not just that it is.
- **C5 · Prompt injection.** The agent reads a document, hits the hidden instruction, visibly does the wrong thing. Keep the payload tame and obviously staged (a marker string, not exfiltration) and say "my own agent, my own sandbox" out loud before running it.

Tier 2 — good demo, needs a purpose-built UI

- **A6 · Speculative decoding.** Two panes, identical prompt, streaming tokens started at once — yours visibly finishes first with identical output. The demo value lives entirely in building this race UI; budget time for it.
- **C6 · AI-text detector.** Hand judges the keyboard, let them try to fool it. Double-edged: it will fail on an unseen generator. Frame that failure as the result, not a bug.
- **C1 · Verified synthetic distillation.** Type a request, a tiny local model emits a regex/SQL query, it executes and passes live. Less dramatic, but the accompanying claim is strong and quantified.
- **C2 · Hinglish eval set.** Five sentences where frontier models misread sarcasm and yours doesn't. Cultural resonance with an Indian panel is underrated as a memorability factor.

Tier 3 — charts, not demos

- **D2, C4, R3, C3** all produce professional-looking results (distribution overlays, calibration curves, hop traces) but nothing that makes a panel lean forward.
- **C3** carries extra risk: judges will critique the generated review comments on quality and it will lose that argument.
- **C4** has one good beat worth staging even inside a chart-heavy talk: ask about something recent, watch the model fabricate with total confidence.

Tier 4 — demo risk

- **D1 · Text diffusion.** The denoising animation — text materializing out of masked tokens step by step — is genuinely beautiful and nobody on the panel will have seen it. But the resulting text will be weak, and a panel that grades on output quality will mark it down regardless of implementation depth. Only viable if your grading rewards technical depth over demo polish.

Demo-day staging rules

- Never demo live off free Colab — it will disconnect, and it will disconnect during your slot.
- Cache every result to disk; run the demo locally off cached artifacts.
- Record a full backup video of the sequence. Panels forgive a video; they do not forgive four minutes of watching you reconnect a runtime.
- Script the demo to a fixed ~90-second beat sequence and rehearse it, including the failure case.

Shortlist — Best for a Live-Demo Panel

Re-ordered for your actual grading format: demo value first, then resume value, all fully compliant and inference-only where possible so nothing can fail to converge the night before.

R2 · RAG corpus poisoning attack/defense

Demo 5 The most theatrical project on the list. Answer flips live when poisoned, flips back when defended. Inference-only — nothing to train, nothing to fail overnight.

R1 · Temporal RAG over Wikipedia revisions

Demo 5 Retrieval that knows documents supersede each other. Dataset comes free from the MediaWiki revision API — gold questions derive mechanically from diffs. Side-by-side baseline comparison is an instant, self-explaining visual. Inference-only.

C5 · Prompt-injection corpus for a tool-using agent

Demo 5 You author the attacks, you own the agent, you measure and then defend. Live attacks land hard with panels; frame explicitly as defensive evaluation of your own system.

C1 · Verified synthetic distillation

Demo 4 Zero scraping, zero ToS risk, zero PII. The yield-rate finding (a large fraction of LLM-generated data is simply wrong) is itself a result, and the live pass/fail execution check is a clean, quick demo beat.

C2 · Hinglish / code-mixed evaluation set

Demo 4 Most distinctive dataset on the list, and the failure examples read well live to an audience that gets the cultural context instantly.

D1 · Discrete text diffusion (not shortlisted for this format)

Highest ceiling on the whole survey and the top resume pick — but wrong format for a live-demo panel.
Demo 2 Output quality is visibly weak by design, and panels grade what they see. Reconsider only if the grading rubric explicitly rewards technical depth over demo polish, or if this is for a report + viva rather than a live run.

Full Catalogue

One entry per idea: what it is, how the dataset is collected, ratings, T4 verdict, the hard part, and the honest weakness.

C1 · Verified synthetic distillation

Complexity 4.5 | Creativity 4.5 | Resume 4.5 | Demo 4

What: Pick a machine-checkable task (NL→regex, NL→SQL, NL→cron, NL→ffmpeg). Generate thousands of pairs with a large model, execute each output to filter for correctness, keep only survivors.

Dataset: Fully self-generated + self-verified. ~10k generated → ~4k survive typically.

T4 verdict: Comfortable — LoRA a 1–3B model on the filtered set.

Hard part: Building the executable verifier per task type is the real engineering, not the prompting.

Demo: Type a request live, the tiny model emits a regex/SQL query, it executes against test cases on screen and passes. Quick, clean, no drama needed.

Weakness: task choice determines everything — pick one where "correct" is unambiguous to check.

C2 · Hinglish / code-mixed evaluation set

Complexity 4 | Creativity 4.5 | Resume 4.5 | Demo 4

What: Scrape code-mixed Indian-language text, hand-annotate for sarcasm/negation/transliteration ambiguity, show model failure, fine-tune to close the gap.

Dataset: YouTube Data API or Reddit API, 1.5–3k hand-annotated examples with reported inter-annotator agreement (Cohen's κ).

T4 verdict: Comfortable.

Hard part: Annotation labor — recruit 2–3 annotators, agree on a rubric first.

Demo: Five sentences where a frontier model misreads sarcasm or negation and yours doesn't. Lands especially well with a panel that gets the cultural context instantly.

Weakness: strip usernames/PII, keep only public content, document the collection protocol explicitly.

C3 · GitHub code-review model

Complexity 4 | Creativity 4 | Resume 4 | Demo 3

What: Mine (diff hunk, review comment) pairs from real PRs, filter noise (LGTM/bots/dupes), fine-tune a reviewer model.

Dataset: GitHub REST API, target 5–15k clean pairs from permissively-licensed (MIT/Apache) repos.

T4 verdict: Comfortable.

Hard part: Matching a comment to the exact hunk it targets; dedup near-identical reviews.

Demo: Feed a live diff, get a generated review comment on screen. Risk: judges will critique the comment's quality directly and it can lose that argument.

Weakness: license hygiene must be documented, not assumed.

C4 · Post-cutoff hallucination benchmark

Complexity 3.5 | Creativity 4 | Resume 4 | Demo 3.5

What: Build a QA set from events after the target model's training cutoff. Measure fabrication confidence, calibration, and how much retrieval fixes it.

Dataset: RSS scraping + hand-written gold answers with source URLs, 300–500 questions.

T4 verdict: Easy, inference-only.

Hard part: None major — this is the most approachable idea on the list.

Demo: Ask about something recent live, watch the model fabricate a confident, wrong answer. One good beat even inside an otherwise chart-heavy talk.

Weakness: smallest engineering surface here. Add a calibration analysis (Brier score, reliability diagram) for depth.

C5 · Prompt-injection corpus for a tool-using agent

Complexity 4 | Creativity 4.5 | Resume 4.5 | Demo 5

What: Build a small tool-using agent, author your own injection attack corpus (direct, indirect-via-retrieval, multi-turn, encoded), measure attack success rate per category, build and evaluate a defense.

Dataset: You author the attacks and harvest the agent's own traces — both original.

T4 verdict: Comfortable, inference-only.

Hard part: Building a genuinely useful (not toy) agent worth attacking.

Demo: Live attack lands, the agent visibly mis-acts (wrong tool call, ignored task) on a hidden instruction. Use a tame, obviously staged payload (a marker string, not exfiltration) and state "my own agent, my own sandbox" out loud before running it.

Weakness: frame explicitly as defensive evaluation of your own system — this framing is what makes it legitimate.

C6 · Cross-generator AI-text detector

Complexity 3.5 | Creativity 4 | Resume 4 | Demo 4.5

What: Collect human text from a niche source you control, generate matched AI counterparts yourself across several models/temperatures, train a detector, then test against a generator held out from training entirely.

Dataset: Both halves self-built and paired by design.

T4 verdict: Easy.

Hard part: Getting genuinely diverse AI-generated samples (multiple models/prompts), not just one generator repeated.

Demo: Hand judges the keyboard, let them try to fool it — the only idea here with audience participation. It will fail on an unseen generator; frame that live failure as the result, not a bug.

Weakness: the interesting result is that detectors collapse out-of-distribution — report that honestly rather than tuning it away.

R1 · Temporal RAG over Wikipedia revisions

Complexity 4 | Creativity 5 | Resume 5 | Demo 5

What: Retrieval-augmented generation that is aware documents supersede each other over time — version-aware filtering, recency reranking, explicit conflict detection at generation.

Dataset: MediaWiki revision API on ~200 articles across their edit history (free, CC-licensed). Candidate questions auto-derived from revision diffs, ~300 hand-verified into a gold set.

T4 verdict: Easy — embeddings + a 3–7B 4-bit reader, no training required.

Hard part: Deriving clean, unambiguous time-qualified questions from raw diffs.

Demo: Side-by-side panes, identical question. Baseline confidently returns the superseded fact; yours returns the current one with a revision timeline rendered underneath — the judge sees why the baseline is wrong, not just that it is.

Weakness: none major — strongest risk/reward ratio on this list, for both grading formats.

R2 · RAG corpus poisoning — attack and defense

Complexity 4 | Creativity 4.5 | Resume 4.5 | Demo 5

What: Build RAG over a scraped corpus, inject adversarial documents crafted to be retrieved and to override correct answers. Measure attack success rate vs. poison count and corpus size, then defend with consistency voting / provenance scoring.

Dataset: Any scraped public-domain corpus + a self-authored poison set (the poison set is the original contribution).

T4 verdict: Easy, inference-only.

Hard part: Crafting poison documents subtle enough to be realistic, not obviously adversarial.

Demo: The most theatrical option on the list. Correct answer → inject poison live → answer flips to the false one → toggle defense on → correct answer returns. Three visible state changes in ~90 seconds.

Weakness: state explicitly this is a defensive evaluation of your own corpus.

R3 · Multi-hop domain RAG (self-scraped corpus)

Complexity 4 | Creativity 3.5 | Resume 3.5 | Demo 3.5

What: Scrape a real domain (court judgments, RBI circulars, university regulations), author ~300 questions requiring 2–3 documents combined, compare flat RAG vs. decomposition vs. graph traversal.

Dataset: Self-scraped domain corpus + hand-authored multi-hop questions.

T4 verdict: Easy.

Hard part: Authoring genuinely multi-hop questions that can't be answered from a single document.

Demo: Show the retrieval trace hopping across documents to assemble an answer flat RAG misses entirely. Decent, not showstopping — the domain choice has to carry interest.

Weakness: least surprising of the RAG ideas — the domain choice has to carry it.

D1 · Discrete text diffusion vs. autoregressive twin

Complexity 5 | Creativity 5 | Resume 5 | Demo 2

What: Implement masked discrete diffusion (D3PM/SEDD-style) for text from scratch. Train an AR transformer of identical parameter count on the same data. Compare quality and parallel-sampling behavior (diffusion denoises all positions at once vs. AR's strictly sequential decode).

Dataset: Scrape a narrow, structurally consistent corpus — recipe steps, commit messages, arXiv abstracts, product descriptions.

T4 verdict: Feasible only if kept small: ~30–60M params, sequence length 64–128, small vocab. A few hours per run — checkpoint to Drive constantly, free Colab disconnects.

Hard part: The diffusion training objective itself; nobody at this level implements it, so no shortcuts to copy from a tutorial.

Demo: The denoising animation (text materializing out of masked tokens) is genuinely striking and rare. But the output text is weak, and a panel grading on live output quality will mark it down regardless of implementation depth — the single biggest resume/demo mismatch in this survey.

Weakness: generated text will be mediocre by design — the deliverable is the controlled comparison, not a good language model. State that upfront in the report as a feature, not an excuse.

D2 · Tabular diffusion + membership-inference attack

Complexity 4 | Creativity 4 | Resume 4 | Demo 3

What: Diffusion model over mixed continuous/categorical tabular data for synthetic generation. Measure utility (train-on-synthetic-test-on-real) and privacy leakage (membership-inference attack against your own generator). Plot the tradeoff.

Dataset: Scrape a tabular corpus yourself — job/rental listings, OpenAQ/CPCB air-quality readings, cricket records.

T4 verdict: Comfortable — tabular models are small, full runs under an hour.

Hard part: Implementing the membership-inference attack correctly, not the diffusion model itself.

Demo: A tradeoff curve and a "was this row in the training set?" guess the audience can play along with. Solid, not a showstopper — it's a chart with one interactive moment.

Weakness: less flashy than text/image diffusion, but the privacy angle is what almost nobody does.

Appendix — Ruled Out by Current Constraints

Kept for the record. Each fails at least one constraint; noted where a straightforward substitution would rescue it.

B1 · Image diffusion from scratch

Fails: text-only | Complexity 4 | Resume 3 | Demo 3

DDPM/U-Net implemented from scratch on a structured image domain (floor plans, circuits, textile motifs). Excluded per the no-images constraint. T4 is tight even at 64×64 resolution.

B2 · Sketch → working artifact

Fails: text-only | Complexity 4 | Resume 5 | Demo 4.5

Hand-drawn ER diagram/wireframe photo → structured IR → generated SQL schema or React component via VLM. Excluded per the no-images constraint, though it has the best live-demo value of anything surveyed — draw on paper, hold it up to a camera, watch working code appear.

A2 · Self-repairing code agent

Fails: readymade data | Complexity 4 | Resume 4 | Demo 3.5

Generate→execute→read traceback→patch→retry loop, benchmarked on HumanEval/MBPP. Rescue: swap the benchmark for a self-collected set of buggy-code/fix pairs mined from your own repos or Stack Overflow, keeping the execution-feedback loop as the core contribution.

A4 · LLM watermarking + attacks

Fails: readymade data | Complexity 4 | Resume 5 | Demo 3.5

Implement green-list watermarking + detector, then break it via paraphrase/temperature/substitution attacks. Originally cited against C4 (streamed, no labels needed) — only fails the constraint if you insist on a curated prompt set; substitute any self-scraped prompt corpus and this becomes fully compliant. Highest resume value on the whole survey after D1.

A5 · Automated red-teaming harness

Fails: readymade data | Complexity 4 | Resume 4.5 | Demo 4

Attacker LLM generates jailbreak attempts against your own small local model, judge scores success, successful attacks mutate and feed back. Rescue: author the seed attack set yourself instead of starting from AdvBench — the mutation loop generates the rest, so the readymade dependency is only at the seed.

A6 · Speculative decoding from scratch

Fails: readymade data (barely applies) | Complexity 4.5 | Resume 3.5 | Demo 4.5

Small draft model proposes tokens, large model verifies via acceptance sampling; benchmark tokens/sec and memory. Barely needs a dataset at all — any prompt set works for latency benchmarking, so this is nearly compliant already. Rarest infra skill surveyed. Revised demo take: build a two-pane live streaming race (identical prompt, both start at once, yours visibly finishes first with identical text) — the race itself is a strong live beat, but the whole demo value depends on building that race UI, and the acceptance-sampling math is easy to get subtly wrong.

A7 · DPO from scratch

Fails: readymade data | Complexity 4.5 | Resume 4 | Demo 2.5

Implement the DPO loss yourself (not via `trl`), fine-tune vs. an SFT-only baseline, compare judge win-rates. Originally cited against hh-rlhf/UltraFeedback. Rescue: generate your own preference pairs by sampling two completions per prompt and ranking them yourself or with a judge model — turns this into a self-collected-data project with the same from-scratch DPO core.

A8 · GraphRAG on HotpotQA

Fails: readymade data | Complexity 4 | Resume 3.5 | Demo 3

Entity/relation extraction into a knowledge graph, multi-hop traversal vs. vector-search baseline. Directly superseded by R3 on this list, which is the same idea with a self-scraped corpus and self-authored questions instead of HotpotQA.

A9 · Self-consistency hallucination detection

Fails: readymade data | Complexity 3.5 | Resume 4 | Demo 3

Sample n responses at temperature, measure self-consistency via NLI entailment + token uncertainty to flag fabrications, score against TruthfulQA/HaluEval labels. Directly superseded by C4 on this list, which is the same detection mechanism validated against a self-collected post-cutoff gold set instead.